VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

AAT REPORT
on

OPERATING SYSTEMS

Submitted by

NISHITA PRASAD (1WA24CS191)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by Nishita Prasad (IWA24CS191), who is bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2024. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Associate Professor
Department of CSE
B. M. S. College of Eng, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
B. M. S. College of Eng, Bengaluru

Index Sheet

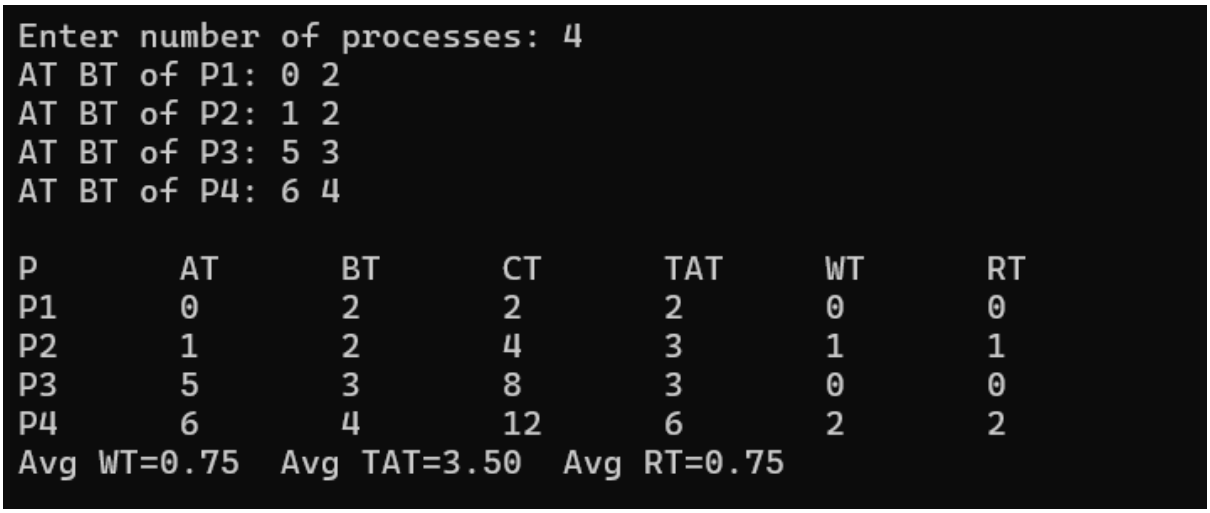
Q1) Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one)
a) FCFS
b) SJF
c) Priority
d) Round Robin (Experiment with different quantum sizes for RR algorithm)

FCFS :

```
#include <stdio.h>
int main() {
   int n;
   printf("Enter number of processes: ");
   scanf("%d", &n);

   int at[n], bt[n], ct[n], tat[n], wt[n], rt[n];

   for(int i=0; i<n; i++){
      printf("AT BT of P%d: ", i+1);
      scanf("%d %d", &at[i], &bt[i]);
      rt[i] = -1;
   }

   int time=0;
   for(int i=0; i<n; i++){
      if(time < at[i]) time = at[i];
      rt[i]  = time - at[i];
      time  += bt[i];
      ct[i]  = time;
      tat[i]  = ct[i] - at[i];
wt[i]   = tat[i] - bt[i];
```

```
    }

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    float tw=0, tt=0, tr=0;
    for(int i=0; i<n; i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
        tw += wt[i]; tt += tat[i]; tr += rt[i];
    }
    printf("Avg WT=%.2f  Avg TAT=%.2f  Avg RT=%.2f\n", tw/n, tt/n, tr/n);
}
```

OUTPUT :

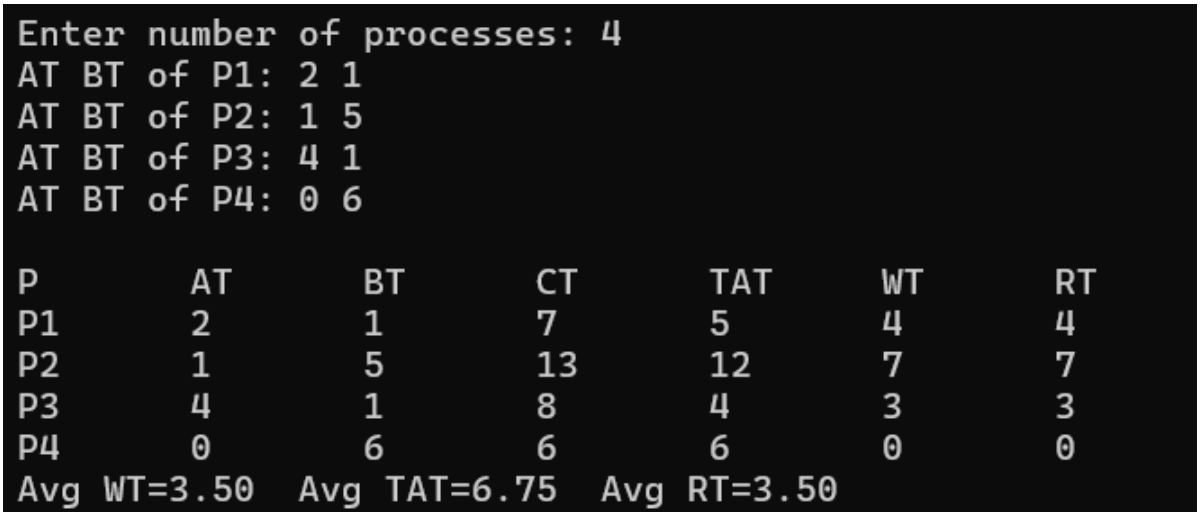


SJF (NON PRE-EMPTIVE) :

```
#include <stdio.h>
int main() {
   int n;
   printf("Enter number of processes: ");
   scanf("%d", &n);

   int at[n], bt[n], ct[n], tat[n], wt[n], rt[n], visited[n];
   for(int i=0; i<n; i++){ visited[i]=0; rt[i]=-1; }

   for(int i=0; i<n; i++){
      printf("AT BT of P%d: ", i+1);
      scanf("%d %d", &at[i], &bt[i]);
   }

   int time=0, completed=0;
   while(completed < n){
      int idx=-1;
      for(int i=0; i<n; i++)
         if(!visited[i] && at[i]<=time){ idx=i; break; }
      if(idx != -1){
         for(int i=0; i<n; i++)
            if(!visited[i] && at[i]<=time && bt[i]<bt[idx]) idx=i;
         rt[idx]  = time - at[idx];
         time    += bt[idx];
         ct[idx]  = time;
         tat[idx] = ct[idx] - at[idx];
         wt[idx]  = tat[idx] - bt[idx];
         visited[idx]=1; completed++;
      }
```

```
        else time++;
    }

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    float tw=0, tt=0, tr=0;
    for(int i=0; i<n; i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
        tw += wt[i]; tt += tat[i]; tr += rt[i];
    }
    printf("Avg WT=%.2f  Avg TAT=%.2f  Avg RT=%.2f\n", tw/n, tt/n, tr/n);
}
```

OUTPUT:



SRTF :

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], rem[n], ct[n], tat[n], wt[n], rt[n], visited[n];
    for(int i=0; i<n; i++){ visited[i]=0; rt[i]=-1; }

    for(int i=0; i<n; i++){
        printf("AT BT of P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rem[i] = bt[i];
    }

    int time=0, completed=0;
    while(completed < n){
        int idx=-1;
        for(int i=0; i<n; i++)
            if(!visited[i] && at[i]<=time){ idx=i; break; }
        if(idx != -1){
            for(int i=0; i<n; i++)
                if(!visited[i] && at[i]<=time && rem[i]<rem[idx]) idx=i;
            if(rt[idx]==-1) rt[idx] = time - at[idx];  // first time running
            rem[idx]--; time++;
            if(rem[idx]==0){
                ct[idx]  = time;
                tat[idx] = ct[idx] - at[idx];
                wt[idx]  = tat[idx] - bt[idx];
```

```
            visited[idx]=1; completed++;
          }
        }
        else time++;
    }

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    float tw=0, tt=0, tr=0;
    for(int i=0; i<n; i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
        tw += wt[i]; tt += tat[i]; tr += rt[i];
    }
    printf("Avg WT=%.2f  Avg TAT=%.2f  Avg RT=%.2f\n", tw/n, tt/n, tr/n);
}
```

OUTPUT :

```
Enter number of processes: 5
AT BT of P1: 2 1
AT BT of P2: 1 5
AT BT of P3: 4 1
AT BT of P4: 0 6
AT BT of P5: 2 3

P       AT      BT      CT      TAT     WT      RT
P1      2       1       3       1       0       0
P2      1       5       11      10      5       0
P3      4       1       5       1       0       0
P4      0       6       16      16      10      0
P5      2       3       7       5       2       1
Avg WT=3.40  Avg TAT=6.60  Avg RT=0.20
```

PRIORITY NON-PREEMPTIVE :

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], pr[n], ct[n], tat[n], wt[n], rt[n], visited[n];
    for(int i=0; i<n; i++){ visited[i]=0; rt[i]=-1; }

    for(int i=0; i<n; i++){
        printf("AT BT PR of P%d: ", i+1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
    }

    int time=0, completed=0;
    while(completed < n){
        int idx=-1;
        for(int i=0; i<n; i++)
            if(!visited[i] && at[i]<=time){ idx=i; break; }
        if(idx != -1){
            for(int i=0; i<n; i++)
                if(!visited[i] && at[i]<=time && pr[i]<pr[idx]) idx=i;
            rt[idx]  = time - at[idx];
            time    += bt[idx];
            ct[idx]  = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx]  = tat[idx] - bt[idx];
            visited[idx]=1; completed++;
```

```
        }
        else time++;
    }

    printf("\nP\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");
    float tw=0, tt=0, tr=0;
    for(int i=0; i<n; i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i], rt[i]);
        tw += wt[i]; tt += tat[i]; tr += rt[i];
    }
    printf("Avg WT=%.2f  Avg TAT=%.2f  Avg RT=%.2f\n", tw/n, tt/n, tr/n);
}
```

OUTPUT :

```
Enter number of processes: 5
AT BT PR of P1: 0 3 5
AT BT PR of P2: 2 2 3
AT BT PR of P3: 3 5 2
AT BT PR of P4: 4 4 4
AT BT PR of P5: 6 1 1

P       AT      BT      PR      CT      TAT     WT      RT
P1      0       3       5       3       3       0       0
P2      2       2       3       11      9       7       7
P3      3       5       2       8       5       0       0
P4      4       4       4       15      11      7       7
P5      6       1       1       9       3       2       2
Avg WT=3.20  Avg TAT=6.20  Avg RT=3.20
```

PRIORITY PRE-EMPTIVE :

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], pr[n], rem[n], ct[n], tat[n], wt[n], rt[n], visited[n];
    for(int i=0; i<n; i++){ visited[i]=0; rt[i]=-1; }

    for(int i=0; i<n; i++){
        printf("AT BT PR of P%d: ", i+1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        rem[i] = bt[i];
    }

    int time=0, completed=0;
    while(completed < n){
        int idx=-1;
        for(int i=0; i<n; i++)
            if(!visited[i] && at[i]<=time){ idx=i; break; }
        if(idx != -1){
            for(int i=0; i<n; i++)
                if(!visited[i] && at[i]<=time && pr[i]<pr[idx]) idx=i;
            if(rt[idx]==-1) rt[idx] = time - at[idx];
            rem[idx]--; time++;
            if(rem[idx]==0){
                ct[idx]  = time;
                tat[idx] = ct[idx] - at[idx];
```

```
            wt[idx]  = tat[idx] - bt[idx];
            visited[idx]=1; completed++;
          }
        }
        else time++;
    }

    printf("\nP\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");
    float tw=0, tt=0, tr=0;
    for(int i=0; i<n; i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i], rt[i]);
        tw += wt[i]; tt += tat[i]; tr += rt[i];
    }
    printf("Avg WT=%.2f  Avg TAT=%.2f  Avg RT=%.2f\n", tw/n, tt/n, tr/n);
}
```

OUTPUT :

```
Enter number of processes: 5
AT BT PR of P1: 0 3 5
AT BT PR of P2: 2 2 3
AT BT PR of P3: 3 5 2
AT BT PR of P4: 4 4 4
AT BT PR of P5: 6 1 1

P       AT      BT      PR      CT      TAT     WT      RT
P1      0       3       5       15      15      12      0
P2      2       2       3       10      8       6       0
P3      3       5       2       9       6       1       0
P4      4       4       4       14      10      6       6
P5      6       1       1       7       1       0       0
Avg WT=5.00  Avg TAT=8.00  Avg RT=1.20
```

ROUND ROBIN :

```
#include <stdio.h>
int main() {
    int n, q;
    printf("Enter number of processes and quantum: ");
    scanf("%d %d", &n, &q);

    int at[n], bt[n], rem[n], ct[n], tat[n], wt[n], rt[n];
    for(int i=0; i<n; i++) rt[i]=-1;

    for(int i=0; i<n; i++){
        printf("AT BT of P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rem[i] = bt[i];
    }

    int queue[100], front=0, rear=0;
    int inqueue[n];
    for(int i=0; i<n; i++) inqueue[i]=0;

    int time=0, completed=0;
    queue[rear++]=0;
    inqueue[0]=1;
    time = at[0];

    while(completed < n){
        int idx = queue[front++];
```

```
if(rt[idx]==-1) rt[idx] = time - at[idx];

if(rem[idx] > q){
    time    += q;
    rem[idx]-= q;
} else {
    time    += rem[idx];
    rem[idx] = 0;
    ct[idx]  = time;
    tat[idx] = ct[idx] - at[idx];
    wt[idx]  = tat[idx] - bt[idx];
    completed++;
}

for(int i=0; i<n; i++)
    if(!inqueue[i] && at[i]<=time){
        queue[rear++]=i;
        inqueue[i]=1;
    }

if(rem[idx]>0)
    queue[rear++]=idx;

if(front==rear){
    int min_at=9999;
    for(int i=0; i<n; i++)
        if(!inqueue[i] && rem[i]>0 && at[i]<min_at)
            min_at=at[i];
    time=min_at;
    for(int i=0; i<n; i++)
```

```
            if(!inqueue[i] && rem[i]>0 && at[i]<=time){
                queue[rear++]=i;
                inqueue[i]=1;
            }
        }
    }

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    float tw=0, tt=0, tr=0;
    for(int i=0; i<n; i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
        tw+=wt[i]; tt+=tat[i]; tr+=rt[i];
    }
    printf("Avg WT=%.2f  Avg TAT=%.2f  Avg RT=%.2f\n", tw/n, tt/n, tr/n);
}
```

OUTPUT :

```
Enter number of processes and quantum: 5 3
AT BT of P1: 0 8
AT BT of P2: 5 2
AT BT of P3: 1 7
AT BT of P4: 6 3
AT BT of P5: 8 5

P       AT      BT      CT      TAT     WT      RT
P1      0       8       22      22      14      0
P2      5       2       11      6       4       4
P3      1       7       23      22      15      2
P4      6       3       14      8       5       5
P5      8       5       25      17      12      9
Avg WT=10.00  Avg TAT=15.00  Avg RT=4.00
```

Q2) Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

MULTILEVEL QUEUE :

```
#include <stdio.h>
int main() {
    int n, i, time=0, q;
    int at[100], bt[100], ct[100], tat[100], wt[100], rem[100], type[100];
    int completed=0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter time quantum: ");
    scanf("%d", &q);

    for(i=0; i<n; i++){
        printf("Enter AT BT Type(0=System 1=User): ");
        scanf("%d %d %d", &at[i], &bt[i], &type[i]);
        rem[i]=bt[i];
    }

    int index=0;
    while(completed < n){
        int executed=0;

        for(i=0; i<n; i++){
            int idx=(index+i)%n;
```

```
    if(type[idx]==0 && at[idx]<=time && rem[idx]>0){
        executed=1;
        if(rem[idx]>q){
            time+=q;
            rem[idx]-=q;
        } else {
            time+=rem[idx];
            rem[idx]=0;
            ct[idx]=time;
            completed++;
        }
        index=(idx+1)%n;
        break;
    }
}

if(!executed){
    for(i=0; i<n; i++){
        if(type[i]==1 && at[i]<=time && rem[i]>0){
            time+=rem[i];
            rem[i]=0;
            ct[i]=time;
            completed++;
            executed=1;
            break;
        }
    }
}

if(!executed) time++;
```

```
    }

    printf("\nAT\tBT\tCT\tTAT\tWT\n");
    float at2=0, aw=0;
    for(i=0; i<n; i++){
        tat[i]=ct[i]-at[i];
        wt[i]=tat[i]-bt[i];
        at2+=tat[i]; aw+=wt[i];
        printf("%d\t%d\t%d\t%d\t%d\n", at[i], bt[i], ct[i], tat[i], wt[i]);
    }
    printf("Avg TAT=%.2f\nAvg WT=%.2f\n", at2/n, aw/n);
}
```

OUTPUT :

```
Enter number of processes: 4
Enter time quantum: 2
Enter AT BT Type(0=System 1=User): 0 3 0
Enter AT BT Type(0=System 1=User): 2 5 1
Enter AT BT Type(0=System 1=User): 5 1 0
Enter AT BT Type(0=System 1=User): 6 5 1

AT      BT      CT      TAT     WT
0       3       3       3       0
2       5       8       6       1
5       1       9       4       3
6       5       14      8       3
Avg TAT=5.25
Avg WT=1.75
```

Q3) Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one)
a) Rate- Monotonic
b) Earliest-deadline First
c) Proportional scheduling

RATE MONOTONIC SCHEDULING

```
#include <stdio.h>
#include <math.h>
#define MAX 10

int gcd(int a, int b){
   while(b){ int t=b; b=a%b; a=t; }
   return a;
}

int lcm(int a, int b){
   return (a*b)/gcd(a,b);
}

int find_lcm(int arr[], int n){
   int res=arr[0];
   for(int i=1; i<n; i++)
      res=lcm(res,arr[i]);
   return res;
}

int main(){
   int n, C[MAX], P[MAX], A[MAX], remaining[MAX];
   for(int i=0; i<MAX; i++) remaining[i]=0;
```

```
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter CPU burst times:\n");
    for(int i=0; i<n; i++) scanf("%d", &C[i]);

    printf("Enter time periods:\n");
    for(int i=0; i<n; i++) scanf("%d", &P[i]);

    printf("Enter arrival times:\n");
    for(int i=0; i<n; i++) scanf("%d", &A[i]);

    int hyper=find_lcm(P,n);
    printf("LCM=%d\n\n", hyper);

    float U=0.0;
    for(int i=0; i<n; i++)
        U+=(float)C[i]/(float)P[i];

    float  bound=(float)n*(pow(2.0,1.0/(float)n)-1.0);

    printf("PID\tBurst\tPeriod\n");
    for(int i=0; i<n; i++)
        printf("%d\t%d\t%d\n", i+1, C[i], P[i]);

    printf("\nU=%f    Bound=%f    =>    %s\n", U, bound, U<=bound ? "Schedulable" : "Not
Schedulable");
    printf("Scheduling for %d ms\n\n", hyper);
    int last=-2;
    for(int t=0; t<hyper; t++){
```

```
        for(int i=0; i<n; i++)
            if(t>=A[i] && (t-A[i])%P[i]==0)
                remaining[i]=C[i];

        int sel=-1, minP=9999;
        for(int i=0; i<n; i++)
            if(remaining[i]>0 && P[i]<minP){ minP=P[i]; sel=i; }

        if(sel != last){
            if(sel==-1) printf("%dms: IDLE\n", t);
            else        printf("%dms: P%d running\n", t, sel+1);
            last=sel;
        }
        if(sel != -1) remaining[sel]--;
    }
}
```

OUTPUT :

```
Enter number of processes: 2
Enter CPU burst times:
20 35
Enter time periods:
50 100
Enter arrival times:
0 0
LCM=100

PID     Burst   Period
1       20      50
2       35      100

U=0.750000  Bound=0.828427  =>  Schedulable
Scheduling for 100 ms

0ms: P1 running
20ms: P2 running
50ms: P1 running
70ms: P2 running
75ms: IDLE
```

EARLIEST DEADLINE FIRST :

```
#include <stdio.h>
#define MAX 10
#define INFINITY 99999
int main(){
   int n;
   int burst[MAX], deadline[MAX], period[MAX];
   int remaining[MAX], next_release[MAX], abs_deadline[MAX];
   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(int i=0; i<n; i++){
      printf("\nProcess %d:\n", i+1);
      printf("Burst time: ");   scanf("%d", &burst[i]);
      printf("Deadline: ");     scanf("%d", &deadline[i]);
      printf("Period: ");       scanf("%d", &period[i]);
      remaining[i]    = burst[i];
      next_release[i] = 0;
      abs_deadline[i] = deadline[i];
   }

   int total_time;
   printf("\nScheduling for how many ms? ");
   scanf("%d", &total_time);
   printf("Scheduling for %d ms\n\n", total_time);

   int time=0;
   while(time < total_time){
      for(int i=0; i<n; i++){
```

```
            if(time==next_release[i] && time!=0){
                remaining[i]    = burst[i];
                abs_deadline[i] = time+deadline[i];
            }
        }
        int min_dl=INFINITY, sel=-1;
        for(int i=0; i<n; i++){
            if(next_release[i]<=time && remaining[i]>0){
                if(abs_deadline[i]<min_dl){
                    min_dl=abs_deadline[i];
                    sel=i;
                }
            }
        }

        if(sel==-1){
            printf("%dms: IDLE\n", time);
        } else {
            printf("%dms: P%d running\n", time, sel+1);
            remaining[sel]--;
            if(remaining[sel]==0)
                next_release[sel]+=period[sel];
        }
        time++;
    }
}
```

OUTPUT :

```
Enter number of processes: 3

Process 1:
Burst time: 3
Deadline: 7
Period: 20

Process 2:
Burst time: 2
Deadline: 4
Period: 5

Process 3:
Burst time: 2
Deadline: 8
Period: 10

Scheduling for how many ms? 20
Scheduling for 20 ms

0ms: P2 running
1ms: P2 running
2ms: P1 running
3ms: P1 running
4ms: P1 running
5ms: P3 running
6ms: P3 running
7ms: P2 running
8ms: P2 running
9ms: IDLE
10ms: P2 running
11ms: P2 running
12ms: P3 running
13ms: P3 running
14ms: IDLE
15ms: P2 running
16ms: P2 running
17ms: IDLE
18ms: IDLE
19ms: IDLE
```

PROPORTIONAL SHARING :

```
#include <stdio.h>

int main(){
    int n, total_time;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], weight[n];
    float share[n];

    for(int i=0; i<n; i++){
        printf("AT and Weight of P%d: ", i+1);
        scanf("%d %d", &at[i], &weight[i]);
    }

    printf("Enter total time: ");
    scanf("%d", &total_time);

    int total_weight=0;
    for(int i=0; i<n; i++) total_weight+=weight[i];

    for(int i=0; i<n; i++)
        share[i]=(float)weight[i]/total_weight * total_time;

    printf("\nProcess\tWeight\tTimeSlice\n");
    for(int i=0; i<n; i++)
        printf("P%d\t%d\t%.2f\n", i+1, weight[i], share[i]);
```

```
    int time=0;
    printf("\nGantt Chart:\n");
    for(int i=0; i<n; i++){
        int slice=(int)share[i];
        for(int j=0; j<slice; j++)
            printf("| P%d ", i+1);
        time+=slice;
    }
    printf("|\n");
    for(int i=0; i<=time; i++) printf("%d   ", i);
    return 0;
}
```

OUTPUT :

```
Enter number of processes: 3
AT and Weight of P1: 0 3
AT and Weight of P2: 0 2
AT and Weight of P3: 0 1
Enter total time: 12

Process Weight  TimeSlice
P1      3       6.00
P2      2       4.00
P3      1       2.00

Gantt Chart:
| P1 | P1 | P1 | P1 | P1 | P1 | P2 | P2 | P2 | P2 | P3 | P3 |
0   1   2   3   4   5   6   7   8   9   10   11   12
```

Q4) Write a C program to simulate:
a) Producer-Consumer problem using semaphores.
b) Dining-Philosopher's problem

PRODUCER-CONSUMER :

```
#include <stdio.h>
#include <stdlib.h>

int mutex=1, full=0, empty=5, x=0;
int buffer[5], in=0, out=0;

int wait(int s)  { return (--s); }
int signal(int s) { return (++s); }

void producer(){
   if(mutex==1 && empty!=0){
      mutex = wait(mutex);
      full  = signal(full);
      empty = wait(empty);
      x++;
      buffer[in]=x;
      printf("\nProducer has produced %d : item %d", in, x);
      in=(in+1)%5;
      mutex=signal(mutex);
   } else {
      printf("\nBuffer is full!");
   }
}

void consumer(){
```

```
    if(mutex==1 && full!=0){
        mutex = wait(mutex);
        full  = wait(full);
        empty = signal(empty);
        int item=buffer[out];
        printf("\nConsumer has consumed %d : item %d", out, item);
        out=(out+1)%5;
        mutex=signal(mutex);
    } else {
        printf("\nBuffer is empty!");
    }
}

int main(){
    int n;
    printf("\n1. Producer\n2. Consumer\n3. Exit");
    while(1){
        printf("\n\nEnter your choice: ");
        scanf("%d", &n);
        switch(n){
            case 1: producer(); break;
            case 2: consumer(); break;
            case 3: printf("Exiting...\n"); exit(0);
            default: printf("Invalid choice!");
        }
    }
    return 0;
}
```

OUTPUT :

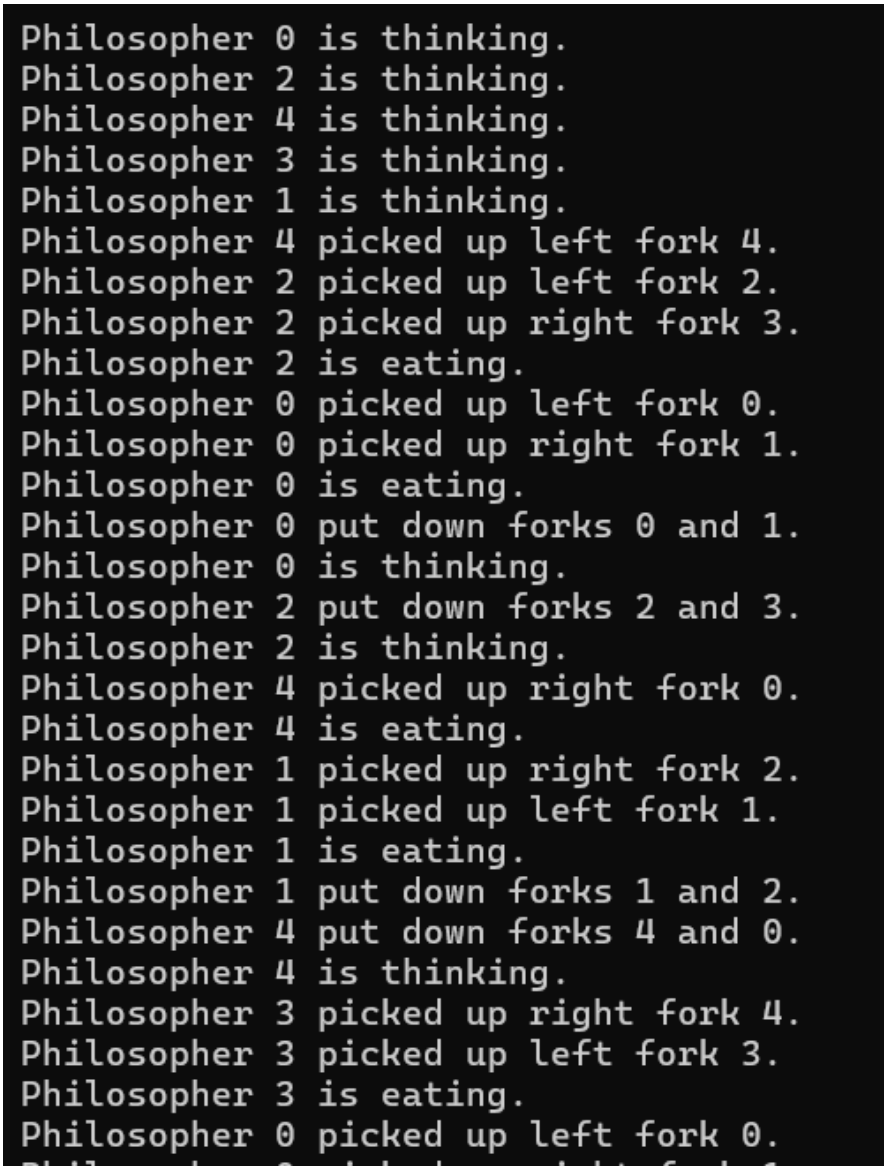
```
1. Producer
2. Consumer
3. Exit

Enter your choice: 1

Producer has produced 0 : item 1

Enter your choice: 1

Producer has produced 1 : item 2

Enter your choice: 1

Producer has produced 2 : item 3

Enter your choice: 2

Consumer has consumed 0 : item 1

Enter your choice: 2

Consumer has consumed 1 : item 2

Enter your choice: 2

Consumer has consumed 2 : item 3

Enter your choice: 2

Buffer is empty!

Enter your choice: 3
Exiting...
```

DINING PHILOSOPHER :

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void* philosopher(void* num){
    int id=*(int*)num;
    int left=id;
    int right=(id+1)%N;
    while(1){
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        if(id%2==0){
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
        } else {
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }
        printf("Philosopher %d is eating.\n", id);
        sleep(2);
```

```
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
    }
    return NULL;
}

int main(){
    int ids[N];
    for(int i=0; i<N; i++){
        pthread_mutex_init(&forks[i], NULL);
        ids[i]=i;
    }
    for(int i=0; i<N; i++)
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    for(int i=0; i<N; i++)
        pthread_join(philosophers[i], NULL);
    for(int i=0; i<N; i++)
        pthread_mutex_destroy(&forks[i]);
    return 0;
}
```

OUTPUT :



and so on…..

Q5) Write a C program to simulate:
a) Bankers' algorithm for the purpose of deadlock avoidance.
b) Deadlock Detection

SAFETY ALGORITHM :

```
#include <stdio.h>
#define MAX 10

int main(){
   int n, m;
   printf("Enter number of processes and resources: ");
   scanf("%d %d", &n, &m);

   int alloc[MAX][MAX], max[MAX][MAX], need[MAX][MAX], avail[MAX];

   printf("Enter Allocation matrix:\n");
   for(int i=0; i<n; i++)
      for(int j=0; j<m; j++)
         scanf("%d", &alloc[i][j]);

   printf("Enter Max matrix:\n");
   for(int i=0; i<n; i++)
      for(int j=0; j<m; j++){
         scanf("%d", &max[i][j]);
         need[i][j]=max[i][j]-alloc[i][j];
      }
   printf("Enter Available resources: ");
   for(int j=0; j<m; j++)
      scanf("%d", &avail[j]);
```

```
    int finish[MAX]={0}, safeseq[MAX], work[MAX];
    for(int j=0; j<m; j++) work[j]=avail[j];

    int count=0;
    while(count < n){
        int found=0;
        for(int i=0; i<n; i++){
            if(!finish[i]){
                int ok=1;
                for(int j=0; j<m; j++)
                    if(need[i][j]>work[j]){ ok=0; break; }
                if(ok){
                    for(int j=0; j<m; j++)
                        work[j]+=alloc[i][j];
                    safeseq[count++]=i;
                    finish[i]=1;
                    found=1;
                }
            }
        }
        if(!found){
            printf("System is in DEADLOCK!\n");
            return 0;
        }
    }
    printf("System is SAFE\nSafe sequence: ");
    for(int i=0; i<n; i++)
        printf("P%d ", safeseq[i]);
    printf("\n");
}
```

OUTPUT :

```
Enter number of processes and resources: 5 3
Enter Allocation matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3
3
Enter Available resources: 3 3 2
System is SAFE
Safe sequence: P1 P3 P4 P0 P2

NITYA PRASAD (1WA24CS194)
Process returned 0 (0x0)   execution time : 57.085 s
Press any key to continue.
```

DEADLOCK DETECTION :

```
#include <stdio.h>
#define MAX 10
int main(){
    int n, m;
    printf("Enter number of processes and resources: ");
    scanf("%d %d", &n, &m);
    int alloc[MAX][MAX], request[MAX][MAX], avail[MAX];
   printf("Enter Allocation matrix:\n");
    for(int i=0; i<n; i++)
        for(int j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);
```

```
printf("Enter Request matrix:\n");
for(int i=0; i<n; i++)
    for(int j=0; j<m; j++)
        scanf("%d", &request[i][j]);

printf("Enter Available resources: ");
for(int j=0; j<m; j++)
    scanf("%d", &avail[j]);

int finish[MAX]={0}, work[MAX];
for(int j=0; j<m; j++) work[j]=avail[j];
for(int i=0; i<n; i++){
    int zero=1;
    for(int j=0; j<m; j++)
        if(alloc[i][j]!=0){ zero=0; break; }
    if(zero) finish[i]=1;
}
int found=1;
while(found){
    found=0;
    for(int i=0; i<n; i++){
        if(!finish[i]){
            int ok=1;
            for(int j=0; j<m; j++)
                if(request[i][j]>work[j]){ ok=0; break; }
            if(ok){
                for(int j=0; j<m; j++)
                    work[j]+=alloc[i][j];
                finish[i]=1;
                found=1;
```

```
            }
        }
    }
}
int deadlock=0;
printf("\nDeadlocked processes: ");
for(int i=0; i<n; i++){
    if(!finish[i]){
        printf("P%d ", i);
        deadlock=1;
    }
}
if(!deadlock)
    printf("None - No Deadlock!");
printf("\n");
}
```

OUTPUT :

```
Enter number of processes and resources: 5 3
Enter Allocation matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
Enter Request matrix:
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2
Enter Available resources: 0 0 0

Deadlocked processes: None - No Deadlock!
```

Q6) Write a C program to simulate the following contiguous

memory allocation techniques.

a) Worst-fit
b) Best-fit
c) First-fit

WORST FIT, BEST FIT, FIRST FIT :

```
#include <stdio.h>
int main(){
   int n, m;
   printf("Enter number of blocks and processes: ");
   scanf("%d %d", &n, &m);

   int block[n], process[m];
   for(int i=0; i<n; i++){ printf("Block %d size: ",i+1); scanf("%d",&block[i]); }
   for(int i=0; i<m; i++){ printf("Process %d size: ",i+1); scanf("%d",&process[i]); }

   int b1[n], b2[n], b3[n], alloc1[m], alloc2[m], alloc3[m];
   for(int i=0; i<n; i++){ b1[i]=block[i]; b2[i]=block[i]; b3[i]=block[i]; }
   for(int i=0; i<m; i++){ alloc1[i]=-1; alloc2[i]=-1; alloc3[i]=-1; }

   for(int i=0; i<m; i++)
      for(int j=0; j<n; j++)
         if(b1[j]>=process[i]){
            alloc1[i]=j;
            b1[j]=-1;    // mark block as used
            break;
         }

   for(int i=0; i<m; i++){
```

```
    int idx=-1;
    for(int j=0; j<n; j++)
        if(b2[j]>=process[i])
            if(idx==-1 || b2[j]<b2[idx]) idx=j;
    if(idx!=-1){
        alloc2[i]=idx;
        b2[idx]=-1;      // mark block as used
    }
}
for(int i=0; i<m; i++){
    int idx=-1;
    for(int j=0; j<n; j++)
        if(b3[j]>=process[i])
            if(idx==-1 || b3[j]>b3[idx]) idx=j;
    if(idx!=-1){
        alloc3[i]=idx;
        b3[idx]=-1;      // mark block as used
    }
}

printf("\n--- First Fit ---\n");
printf("Process\tSize\tBlock\n");
for(int i=0; i<m; i++){
    if(alloc1[i]==-1) printf("P%d\t%d\tNA\n", i+1, process[i]);
    else              printf("P%d\t%d\tBlock %d\n", i+1, process[i], alloc1[i]+1);
}

printf("\n--- Best Fit ---\n");
printf("Process\tSize\tBlock\n");
for(int i=0; i<m; i++){
```

```
        if(alloc2[i]==-1) printf("P%d\t%d\tNA\n", i+1, process[i]);
        else              printf("P%d\t%d\tBlock %d\n", i+1, process[i], alloc2[i]+1);
    }

    printf("\n--- Worst Fit ---\n");
    printf("Process\tSize\tBlock\n");
    for(int i=0; i<m; i++){
        if(alloc3[i]==-1) printf("P%d\t%d\tNA\n", i+1, process[i]);
        else              printf("P%d\t%d\tBlock %d\n", i+1, process[i], alloc3[i]+1);
    }
}
```

OUTPUT :

```
Enter number of blocks and processes: 5 4
Block 1 size: 100
Block 2 size: 500
Block 3 size: 200
Block 4 size: 300
Block 5 size: 600
Process 1 size: 212
Process 2 size: 417
Process 3 size: 112
Process 4 size: 426

--- First Fit ---
Process Size    Block
P1      212     Block 2
P2      417     Block 5
P3      112     Block 3
P4      426     NA

--- Best Fit ---
Process Size    Block
P1      212     Block 4
P2      417     Block 2
P3      112     Block 3
P4      426     Block 5

--- Worst Fit ---
Process Size    Block
P1      212     Block 5
P2      417     Block 2
P3      112     Block 4
P4      426     NA

Process returned 0 (0x0)   execution time : 27.318 s
Press any key to continue.
```

Q7) Write a C program to simulate page replacement algorithms.

(Any one)
a) FIFO
b) LRU
c) Optimal

PAGE REPLACEMENT :

```
#include <stdio.h>
int main(){
    int n, f;
    printf("Enter number of pages and frames: ");
    scanf("%d %d", &n, &f);

    int pages[n];
    printf("Enter page reference string: ");
    for(int i=0; i<n; i++) scanf("%d",&pages[i]);
    int frames[f], faults, pos;
    for(int i=0; i<f; i++) frames[i]=-1;
    faults=0; pos=0;
    printf("\n--- FIFO ---\n");
    for(int i=0; i<n; i++){
        int found=0;
        for(int j=0; j<f; j++) if(frames[j]==pages[i]){ found=1; break; }
        if(!found){
            faults++;
            frames[pos]=pages[i];
            pos=(pos+1)%f;
            printf("Page %d -> ", pages[i]);
            for(int j=0; j<f; j++) frames[j]==-1?printf("- "):printf("%d ",frames[j]);
            printf("(FAULT)\n");
        } else {
```

```
        printf("Page %d -> ", pages[i]);
        for(int j=0; j<f; j++) frames[j]==-1?printf("- "):printf("%d ",frames[j]);
        printf("\n");
    }
}
printf("Total Faults = %d\n", faults);
for(int i=0; i<f; i++) frames[i]=-1;
faults=0;
printf("\n--- LRU ---\n");
for(int i=0; i<n; i++){
    int found=0;
    for(int j=0; j<f; j++) if(frames[j]==pages[i]){ found=1; break; }
    if(!found){
        faults++;
        int idx=0, last=i;
        for(int j=0; j<f; j++){
            if(frames[j]==-1){ idx=j; break; }
            int k;
            for(k=i-1; k>=0; k--) if(frames[j]==pages[k]) break;
            if(k<last){ last=k; idx=j; }
        }
        frames[idx]=pages[i];
        printf("Page %d -> ", pages[i]);
        for(int j=0; j<f; j++) frames[j]==-1?printf("- "):printf("%d ",frames[j]);
        printf("(FAULT)\n");
    } else {
        printf("Page %d -> ", pages[i]);
        for(int j=0; j<f; j++) frames[j]==-1?printf("- "):printf("%d ",frames[j]);
        printf("\n");
    }
```

```
    }
    printf("Total Faults = %d\n", faults);
    for(int i=0; i<f; i++) frames[i]=-1;
    faults=0;
    printf("\n--- Optimal ---\n");
    for(int i=0; i<n; i++){
        int found=0;
        for(int j=0; j<f; j++) if(frames[j]==pages[i]){ found=1; break; }
        if(!found){
            faults++;
            int idx=0, farthest=i;
            for(int j=0; j<f; j++){
                if(frames[j]==-1){ idx=j; break; }
                int k;
                for(k=i+1; k<n; k++) if(frames[j]==pages[k]) break;
                if(k>farthest){ farthest=k; idx=j; }
            }
            frames[idx]=pages[i];
            printf("Page %d -> ", pages[i]);
            for(int j=0; j<f; j++) frames[j]==-1?printf("- "):printf("%d ",frames[j]);
            printf("(FAULT)\n");
        } else {
            printf("Page %d -> ", pages[i]);
            for(int j=0; j<f; j++) frames[j]==-1?printf("- "):printf("%d ",frames[j]);
            printf("\n");
        }
    }
    printf("Total Faults = %d\n", faults);
}
```

OUTPUT :

```
Enter number of pages and frames: 12 3
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3

--- FIFO ---
Page 7 -> 7 - - (FAULT)
Page 0 -> 7 0 - (FAULT)
Page 1 -> 7 0 1 (FAULT)
Page 2 -> 2 0 1 (FAULT)
Page 0 -> 2 0 1
Page 3 -> 2 3 1 (FAULT)
Page 0 -> 2 3 0 (FAULT)
Page 4 -> 4 3 0 (FAULT)
Page 2 -> 4 2 0 (FAULT)
Page 3 -> 4 2 3 (FAULT)
Page 0 -> 0 2 3 (FAULT)
Page 3 -> 0 2 3
Total Faults = 10

--- LRU ---
Page 7 -> 7 - - (FAULT)
Page 0 -> 7 0 - (FAULT)
Page 1 -> 7 0 1 (FAULT)
Page 2 -> 2 0 1 (FAULT)
Page 0 -> 2 0 1
Page 3 -> 2 0 3 (FAULT)
Page 0 -> 2 0 3
Page 4 -> 4 0 3 (FAULT)
Page 2 -> 4 0 2 (FAULT)
Page 3 -> 4 3 2 (FAULT)
Page 0 -> 0 3 2 (FAULT)
Page 3 -> 0 3 2
Total Faults = 9

--- Optimal ---
Page 7 -> 7 - - (FAULT)
Page 0 -> 7 0 - (FAULT)
Page 1 -> 7 0 1 (FAULT)
Page 2 -> 2 0 1 (FAULT)
Page 0 -> 2 0 1
Page 3 -> 2 0 3 (FAULT)
Page 0 -> 2 0 3
Page 4 -> 2 4 3 (FAULT)
Page 2 -> 2 4 3
Page 3 -> 2 4 3
Page 0 -> 0 4 3 (FAULT)
Page 3 -> 0 4 3
Total Faults = 7
```